```
            }
            if (j == n) ans=bip;
        }
        ans *= i0(x)/bi;                    Normalize with bessi0.
        return x < 0.0 && (n & 1) ? -ans : ans;
    }
}
```

The function `ldexp`, used above, is a standard C and C++ library function for scaling the binary exponent of a number.

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at `http://www.nr.com/aands`, Chapter 9.

Carrier, G.F., Krook, M. and Pearson, C.E. 1966, *Functions of a Complex Variable* (New York: McGraw-Hill), pp. 220ff.

SPECFUN, 2007+, at `http://www.netlib.org/specfun`.[1]

Hart, J.F., et al. 1968, *Computer Approximations* (New York: Wiley), §6.8, p. 141.[2]

Numerical Recipes Software 2007, "Coefficients Used in the Bessjy and Bessik Objects," *Numerical Recipes Webnote No. 7*, at `http://www.nr.com/webnotes?7` [3]

6.6 Bessel Functions of Fractional Order, Airy Functions, Spherical Bessel Functions

Many algorithms have been proposed for computing Bessel functions of fractional order numerically. Most of them are, in fact, not very good in practice. The routines given here are rather complicated, but they can be recommended wholeheartedly.

6.6.1 Ordinary Bessel Functions

The basic idea is *Steed's method*, which was originally developed [1] for Coulomb wave functions. The method calculates J_ν, J'_ν, Y_ν, and Y'_ν simultaneously, and so involves four relations among these functions. Three of the relations come from two continued fractions, one of which is complex. The fourth is provided by the Wronskian relation

$$W \equiv J_\nu Y'_\nu - Y_\nu J'_\nu = \frac{2}{\pi x} \tag{6.6.1}$$

The first continued fraction, CF1, is defined by

$$\begin{aligned} f_\nu \equiv \frac{J'_\nu}{J_\nu} &= \frac{\nu}{x} - \frac{J_{\nu+1}}{J_\nu} \\ &= \frac{\nu}{x} - \frac{1}{2(\nu+1)/x -} \; \frac{1}{2(\nu+2)/x -} \cdots \end{aligned} \tag{6.6.2}$$

You can easily derive it from the three-term recurrence relation for Bessel functions: Start with equation (6.5.6) and use equation (5.4.18). Forward evaluation of the continued fraction by one of the methods of §5.2 is essentially equivalent to backward recurrence of the recurrence relation. The rate of convergence of CF1 is determined by the position of the *turning point* $x_{\rm tp} = \sqrt{\nu(\nu+1)} \approx \nu$, beyond which the Bessel functions become oscillatory. If $x \lesssim x_{\rm tp}$,

convergence is very rapid. If $x \gtrsim x_{\rm tp}$, then each iteration of the continued fraction effectively increases ν by one until $x \lesssim x_{\rm tp}$; thereafter rapid convergence sets in. Thus the number of iterations of CF1 is of order x for large x. In the routine `besseljy` we set the maximum allowed number of iterations to 10,000. For larger x, you can use the usual asymptotic expressions for Bessel functions.

One can show that the sign of J_ν is the same as the sign of the denominator of CF1 once it has converged.

The complex continued fraction CF2 is defined by

$$p + iq \equiv \frac{J_\nu' + iY_\nu'}{J_\nu + iY_\nu} = -\frac{1}{2x} + i + \frac{i}{x}\,\frac{(1/2)^2 - \nu^2}{2(x+i) +}\,\frac{(3/2)^2 - \nu^2}{2(x+2i) +}\cdots \tag{6.6.3}$$

(We sketch the derivation of CF2 in the analogous case of modified Bessel functions in the next subsection.) This continued fraction converges rapidly for $x \gtrsim x_{\rm tp}$, while convergence fails as $x \to 0$. We have to adopt a special method for small x, which we describe below. For x not too small, we can ensure that $x \gtrsim x_{\rm tp}$ by a stable recurrence of J_ν and J_ν' downward to a value $\nu = \mu \lesssim x$, thus yielding the ratio f_μ at this lower value of ν. This is the stable direction for the recurrence relation. The initial values for the recurrence are

$$J_\nu = \text{arbitrary}, \qquad J_\nu' = f_\nu J_\nu, \tag{6.6.4}$$

with the sign of the arbitrary initial value of J_ν chosen to be the sign of the denominator of CF1. Choosing the initial value of J_ν very small minimizes the possibility of overflow during the recurrence. The recurrence relations are

$$\begin{aligned} J_{\nu-1} &= \frac{\nu}{x} J_\nu + J_\nu' \\ J_{\nu-1}' &= \frac{\nu - 1}{x} J_{\nu-1} - J_\nu \end{aligned} \tag{6.6.5}$$

Once CF2 has been evaluated at $\nu = \mu$, then with the Wronskian (6.6.1) we have enough relations to solve for all four quantities. The formulas are simplified by introducing the quantity

$$\gamma \equiv \frac{p - f_\mu}{q} \tag{6.6.6}$$

Then

$$J_\mu = \pm \left(\frac{W}{q + \gamma(p - f_\mu)} \right)^{1/2} \tag{6.6.7}$$

$$J_\mu' = f_\mu J_\mu \tag{6.6.8}$$

$$Y_\mu = \gamma J_\mu \tag{6.6.9}$$

$$Y_\mu' = Y_\mu \left(p + \frac{q}{\gamma} \right) \tag{6.6.10}$$

The sign of J_μ in (6.6.7) is chosen to be the same as the sign of the initial J_ν in (6.6.4).

Once all four functions have been determined at the value $\nu = \mu$, we can find them at the original value of ν. For J_ν and J_ν', simply scale the values in (6.6.4) by the ratio of (6.6.7) to the value found after applying the recurrence (6.6.5). The quantities Y_ν and Y_ν' can be found by starting with the values in (6.6.9) and (6.6.10) and using the stable upward recurrence

$$Y_{\nu+1} = \frac{2\nu}{x} Y_\nu - Y_{\nu-1} \tag{6.6.11}$$

together with the relation

$$Y_\nu' = \frac{\nu}{x} Y_\nu - Y_{\nu+1} \tag{6.6.12}$$

Now turn to the case of small x, when CF2 is not suitable. Temme [2] has given a good method of evaluating Y_ν and $Y_{\nu+1}$, and hence Y_ν' from (6.6.12), by series expansions that

accurately handle the singularity as $x \to 0$. The expansions work only for $|\nu| \le 1/2$, and so now the recurrence (6.6.5) is used to evaluate f_ν at a value $\nu = \mu$ in this interval. Then one calculates J_μ from

$$J_\mu = \frac{W}{Y'_\mu - Y_\mu f_\mu} \tag{6.6.13}$$

and J'_μ from (6.6.8). The values at the original value of ν are determined by scaling as before, and the Y's are recurred up as before.

Temme's series are

$$Y_\nu = -\sum_{k=0}^{\infty} c_k g_k \qquad Y_{\nu+1} = -\frac{2}{x}\sum_{k=0}^{\infty} c_k h_k \tag{6.6.14}$$

Here

$$c_k = \frac{(-x^2/4)^k}{k!} \tag{6.6.15}$$

while the coefficients g_k and h_k are defined in terms of quantities p_k, q_k, and f_k that can be found by recursion:

$$\begin{aligned} g_k &= f_k + \frac{2}{\nu}\sin^2\left(\frac{\nu\pi}{2}\right) q_k \\ h_k &= -k g_k + p_k \\ p_k &= \frac{p_{k-1}}{k-\nu} \\ q_k &= \frac{q_{k-1}}{k+\nu} \\ f_k &= \frac{k f_{k-1} + p_{k-1} + q_{k-1}}{k^2 - \nu^2} \end{aligned} \tag{6.6.16}$$

The initial values for the recurrences are

$$\begin{aligned} p_0 &= \frac{1}{\pi}\left(\frac{x}{2}\right)^{-\nu}\Gamma(1+\nu) \\ q_0 &= \frac{1}{\pi}\left(\frac{x}{2}\right)^{\nu}\Gamma(1-\nu) \\ f_0 &= \frac{2}{\pi}\frac{\nu\pi}{\sin\nu\pi}\left[\cosh\sigma\,\Gamma_1(\nu) + \frac{\sinh\sigma}{\sigma}\ln\left(\frac{2}{x}\right)\Gamma_2(\nu)\right] \end{aligned} \tag{6.6.17}$$

with

$$\begin{aligned} \sigma &= \nu\ln\left(\frac{2}{x}\right) \\ \Gamma_1(\nu) &= \frac{1}{2\nu}\left[\frac{1}{\Gamma(1-\nu)} - \frac{1}{\Gamma(1+\nu)}\right] \\ \Gamma_2(\nu) &= \frac{1}{2}\left[\frac{1}{\Gamma(1-\nu)} + \frac{1}{\Gamma(1+\nu)}\right] \end{aligned} \tag{6.6.18}$$

The whole point of writing the formulas in this way is that the potential problems as $\nu \to 0$ can be controlled by evaluating $\nu\pi/\sin\nu\pi$, $\sinh\sigma/\sigma$, and Γ_1 carefully. In particular, Temme gives Chebyshev expansions for $\Gamma_1(\nu)$ and $\Gamma_2(\nu)$. We have rearranged his expansion for Γ_1 to be explicitly an even series in ν for more efficient evaluation, as explained in §5.8.

Because J_ν, Y_ν, J'_ν, and Y'_ν are all calculated simultaneously, a single `void` function sets them all. You then grab those that you need directly from the object. Alternatively, the functions `jnu` and `ynu` can be used. (We've omitted similar helper functions for the derivatives, but you can easily add them.) The object `Bessel` contains various other methods that will be discussed below.

The routines assume $\nu \geq 0$. For negative ν you can use the reflection formulas

$$\begin{aligned} J_{-\nu} &= \cos \nu\pi \; J_\nu - \sin \nu\pi \; Y_\nu \\ Y_{-\nu} &= \sin \nu\pi \; J_\nu + \cos \nu\pi \; Y_\nu \end{aligned} \tag{6.6.19}$$

The routine also assumes $x > 0$. For $x < 0$, the functions are in general complex but expressible in terms of functions with $x > 0$. For $x = 0$, Y_ν is singular. The complex arithmetic is carried out explicitly with real variables.

besselfrac.h

```
struct Bessel {
```

Object for Bessel functions of arbitrary order ν, and related functions.

```
    static const Int NUSE1=7, NUSE2=8;
    static const Doub c1[NUSE1],c2[NUSE2];
    Doub xo,nuo;                          Saved x and ν from last call.
    Doub jo,yo,jpo,ypo;                   Set by besseljy.
    Doub io,ko,ipo,kpo;                   Set by besselik.
    Doub aio,bio,aipo,bipo;               Set by airy.
    Doub sphjo,sphyo,sphjpo,sphypo;       Set by sphbes.
    Int sphno;

    Bessel() : xo(9.99e99), nuo(9.99e99), sphno(-9999) {}
```

Default constructor. No arguments.

```
    void besseljy(const Doub nu, const Doub x);
```

Calculate Bessel functions $J_\nu(x)$ and $Y_\nu(x)$ and their derivatives.

```
    void besselik(const Doub nu, const Doub x);
```

Calculate Bessel functions $I_\nu(x)$ and $K_\nu(x)$ and their derivatives.

```
    Doub jnu(const Doub nu, const Doub x) {
```

Simple interface returning $J_\nu(x)$.

```
        if (nu != nuo || x != xo) besseljy(nu,x);
        return jo;
    }
    Doub ynu(const Doub nu, const Doub x) {
```

Simple interface returning $Y_\nu(x)$.

```
        if (nu != nuo || x != xo) besseljy(nu,x);
        return yo;
    }
    Doub inu(const Doub nu, const Doub x) {
```

Simple interface returning $I_\nu(x)$.

```
        if (nu != nuo || x != xo) besselik(nu,x);
        return io;
    }
    Doub knu(const Doub nu, const Doub x) {
```

Simple interface returning $K_\nu(x)$.

```
        if (nu != nuo || x != xo) besselik(nu,x);
        return ko;
    }

    void airy(const Doub x);
```

Calculate Airy functions $\mathrm{Ai}(x)$ and $\mathrm{Bi}(x)$ and their derivatives.

```
    Doub airy_ai(const Doub x);
```

Simple interface returning $\mathrm{Ai}(x)$.

```
    Doub airy_bi(const Doub x);
```

Simple interface returning $\mathrm{Bi}(x)$.

```
    void sphbes(const Int n, const Doub x);
```

Calculate spherical Bessel functions $j_n(x)$ and $y_n(x)$ and their derivatives.

```
    Doub sphbesj(const Int n, const Doub x);
```

Simple interface returning $j_n(x)$.

```
    Doub sphbesy(const Int n, const Doub x);
```

Simple interface returning $y_n(x)$.

```
    inline Doub chebev(const Doub *c, const Int m, const Doub x) {
    Utility used by besseljy and besselik, evaluates Chebyshev series.
        Doub d=0.0,dd=0.0,sv;
        Int j;
        for (j=m-1;j>0;j--) {
            sv=d;
            d=2.*x*d-dd+c[j];
            dd=sv;
        }
        return x*d-dd+0.5*c[0];
    }
};

const Doub Bessel::c1[7] = {-1.142022680371168e0,6.5165112670737e-3,
    3.087090173086e-4,-3.4706269649e-6,6.9437664e-9,3.67795e-11,
    -1.356e-13};
const Doub Bessel::c2[8] = {1.843740587300905e0,-7.68528408447867e-2,
    1.2719271366546e-3,-4.9717367042e-6,-3.31261198e-8,2.423096e-10,
    -1.702e-13,-1.49e-15};
```

The code listing for `Bessel::besseljy` is in a Webnote [4].

6.6.2 Modified Bessel Functions

Steed's method does not work for modified Bessel functions because in this case CF2 is purely imaginary and we have only three relations among the four functions. Temme [3] has given a normalization condition that provides the fourth relation.

The Wronskian relation is

$$W \equiv I_\nu K'_\nu - K_\nu I'_\nu = -\frac{1}{x} \tag{6.6.20}$$

The continued fraction CF1 becomes

$$f_\nu \equiv \frac{I'_\nu}{I_\nu} = \frac{\nu}{x} + \frac{1}{2(\nu+1)/x +} \; \frac{1}{2(\nu+2)/x +} \cdots \tag{6.6.21}$$

To get CF2 and the normalization condition in a convenient form, consider the sequence of confluent hypergeometric functions

$$z_n(x) = U(\nu + 1/2 + n, 2\nu + 1, 2x) \tag{6.6.22}$$

for fixed ν. Then

$$K_\nu(x) = \pi^{1/2}(2x)^\nu e^{-x} z_0(x) \tag{6.6.23}$$

$$\frac{K_{\nu+1}(x)}{K_\nu(x)} = \frac{1}{x}\left[\nu + \frac{1}{2} + x + \left(\nu^2 - \frac{1}{4}\right)\frac{z_1}{z_0}\right] \tag{6.6.24}$$

Equation (6.6.23) is the standard expression for K_ν in terms of a confluent hypergeometric function, while equation (6.6.24) follows from relations between contiguous confluent hypergeometric functions (equations 13.4.16 and 13.4.18 in Ref. [5]). Now the functions z_n satisfy the three-term recurrence relation (equation 13.4.15 in Ref. [5])

$$z_{n-1}(x) = b_n z_n(x) + a_{n+1} z_{n+1} \tag{6.6.25}$$

with

$$\begin{aligned} b_n &= 2(n + x) \\ a_{n+1} &= -[(n + 1/2)^2 - \nu^2] \end{aligned} \tag{6.6.26}$$

Following the steps leading to equation (5.4.18), we get the continued fraction CF2

$$\frac{z_1}{z_0} = \frac{1}{b_1 +} \; \frac{a_2}{b_2 +} \cdots \tag{6.6.27}$$

from which (6.6.24) gives $K_{\nu+1}/K_\nu$ and thus K'_ν/K_ν.

Temme's normalization condition is that

$$\sum_{n=0}^{\infty} C_n z_n = \left(\frac{1}{2x}\right)^{\nu+1/2} \tag{6.6.28}$$

where

$$C_n = \frac{(-1)^n}{n!}\frac{\Gamma(\nu+1/2+n)}{\Gamma(\nu+1/2-n)} \tag{6.6.29}$$

Note that the C_n's can be determined by recursion:

$$C_0 = 1, \qquad C_{n+1} = -\frac{a_{n+1}}{n+1}C_n \tag{6.6.30}$$

We use the condition (6.6.28) by finding

$$S = \sum_{n=1}^{\infty} C_n \frac{z_n}{z_0} \tag{6.6.31}$$

Then

$$z_0 = \left(\frac{1}{2x}\right)^{\nu+1/2} \frac{1}{1+S} \tag{6.6.32}$$

and (6.6.23) gives K_ν.

Thompson and Barnett [6] have given a clever method of doing the sum (6.6.31) simultaneously with the forward evaluation of the continued fraction CF2. Suppose the continued fraction is being evaluated as

$$\frac{z_1}{z_0} = \sum_{n=0}^{\infty} \Delta h_n \tag{6.6.33}$$

where the increments Δh_n are being found by, e.g., Steed's algorithm or the modified Lentz's algorithm of §5.2. Then the approximation to S keeping the first N terms can be found as

$$S_N = \sum_{n=1}^{N} Q_n \Delta h_n \tag{6.6.34}$$

Here

$$Q_n = \sum_{k=1}^{n} C_k q_k \tag{6.6.35}$$

and q_k is found by recursion from

$$q_{k+1} = (q_{k-1} - b_k q_k)/a_{k+1} \tag{6.6.36}$$

starting with $q_0 = 0$, $q_1 = 1$. For the case at hand, approximately three times as many terms are needed to get S to converge as are needed simply for CF2 to converge.

To find K_ν and $K_{\nu+1}$ for small x we use series analogous to (6.6.14):

$$K_\nu = \sum_{k=0}^{\infty} c_k f_k \qquad K_{\nu+1} = \frac{2}{x}\sum_{k=0}^{\infty} c_k h_k \tag{6.6.37}$$

Here

$$\begin{aligned} c_k &= \frac{(x^2/4)^k}{k!} \\ h_k &= -kf_k + p_k \\ p_k &= \frac{p_{k-1}}{k-\nu} \\ q_k &= \frac{q_{k-1}}{k+\nu} \\ f_k &= \frac{kf_{k-1} + p_{k-1} + q_{k-1}}{k^2 - \nu^2} \end{aligned} \tag{6.6.38}$$

The initial values for the recurrences are

$$\begin{aligned} p_0 &= \frac{1}{2}\left(\frac{x}{2}\right)^{-\nu} \Gamma(1+\nu) \\ q_0 &= \frac{1}{2}\left(\frac{x}{2}\right)^{\nu} \Gamma(1-\nu) \\ f_0 &= \frac{\nu\pi}{\sin\nu\pi}\left[\cosh\sigma\,\Gamma_1(\nu) + \frac{\sinh\sigma}{\sigma}\ln\left(\frac{2}{x}\right)\Gamma_2(\nu)\right] \end{aligned} \tag{6.6.39}$$

Both the series for small x, and CF2 and the normalization relation (6.6.28) require $|\nu| \le 1/2$. In both cases, therefore, we recurse I_ν down to a value $\nu = \mu$ in this interval, find K_μ there, and recurse K_ν back up to the original value of ν.

The routine assumes $\nu \ge 0$. For negative ν use the reflection formulas

$$\begin{aligned} I_{-\nu} &= I_\nu + \frac{2}{\pi}\sin(\nu\pi)\,K_\nu \\ K_{-\nu} &= K_\nu \end{aligned} \tag{6.6.40}$$

Note that for large x, $I_\nu \sim e^x$ and $K_\nu \sim e^{-x}$, and so these functions will overflow or underflow. It is often desirable to be able to compute the scaled quantities $e^{-x}I_\nu$ and $e^x K_\nu$. Simply omitting the factor e^{-x} in equation (6.6.23) will ensure that all four quantities will have the appropriate scaling. If you also want to scale the four quantities for small x when the series in equation (6.6.37) are used, you must multiply each series by e^x.

As with `besseljy`, you can either call the `void` function `besselik`, and then retrieve the function and/or derivative values from the object, or else just call `inu` or `knu`.

The code listing for `Bessel::besselik` is in a Webnote [4].

6.6.3 Airy Functions

For positive x, the Airy functions are defined by

$$\mathrm{Ai}(x) = \frac{1}{\pi}\sqrt{\frac{x}{3}}K_{1/3}(z) \tag{6.6.41}$$

$$\mathrm{Bi}(x) = \sqrt{\frac{x}{3}}[I_{1/3}(z) + I_{-1/3}(z)] \tag{6.6.42}$$

where

$$z = \frac{2}{3}x^{3/2} \tag{6.6.43}$$

By using the reflection formula (6.6.40), we can convert (6.6.42) into the computationally more useful form

$$\mathrm{Bi}(x) = \sqrt{x}\left[\frac{2}{\sqrt{3}}I_{1/3}(z) + \frac{1}{\pi}K_{1/3}(z)\right] \tag{6.6.44}$$

so that Ai and Bi can be evaluated with a single call to `besselik`.

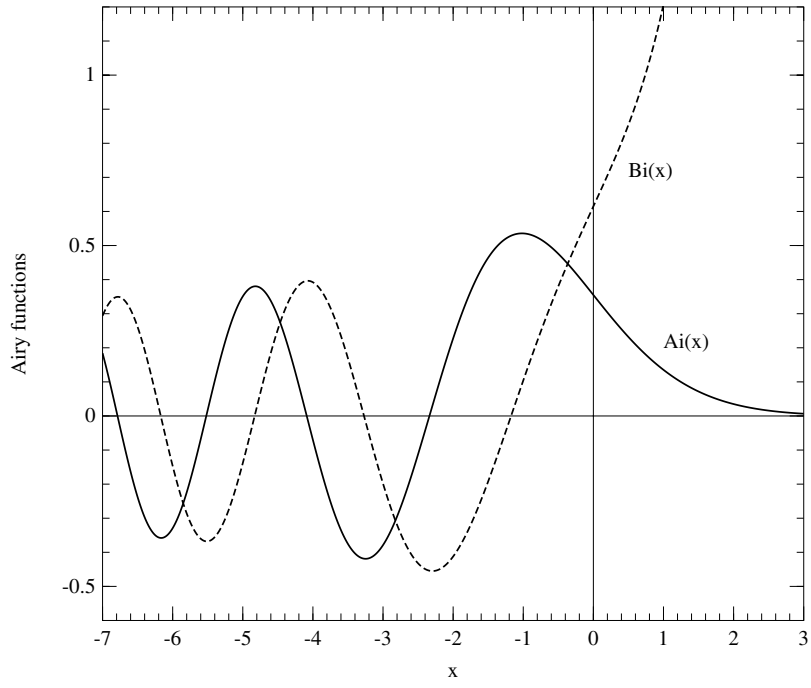

Figure 6.6.1. Airy functions $\mathrm{Ai}(x)$ and $\mathrm{Bi}(x)$.

The derivatives should not be evaluated by simply differentiating the above expressions because of possible subtraction errors near $x = 0$. Instead, use the equivalent expressions

$$\begin{aligned} \mathrm{Ai}'(x) &= -\frac{x}{\pi\sqrt{3}} K_{2/3}(z) \\ \mathrm{Bi}'(x) &= x\left[\frac{2}{\sqrt{3}} I_{2/3}(z) + \frac{1}{\pi} K_{2/3}(z)\right] \end{aligned} \tag{6.6.45}$$

The corresponding formulas for negative arguments are

$$\begin{aligned} \mathrm{Ai}(-x) &= \frac{\sqrt{x}}{2}\left[J_{1/3}(z) - \frac{1}{\sqrt{3}} Y_{1/3}(z)\right] \\ \mathrm{Bi}(-x) &= -\frac{\sqrt{x}}{2}\left[\frac{1}{\sqrt{3}} J_{1/3}(z) + Y_{1/3}(z)\right] \\ \mathrm{Ai}'(-x) &= \frac{x}{2}\left[J_{2/3}(z) + \frac{1}{\sqrt{3}} Y_{2/3}(z)\right] \\ \mathrm{Bi}'(-x) &= \frac{x}{2}\left[\frac{1}{\sqrt{3}} J_{2/3}(z) - Y_{2/3}(z)\right] \end{aligned} \tag{6.6.46}$$

besselfrac.h

```
void Bessel::airy(const Doub x) {
```

Sets aio, bio, aipo, and bipo, respectively, to the Airy functions $\mathrm{Ai}(x)$, $\mathrm{Bi}(x)$ and their derivatives $\mathrm{Ai}'(x)$, $\mathrm{Bi}'(x)$.

```
    static const Doub PI=3.141592653589793238,
        ONOVRT=0.577350269189626,THR=1./3.,TWOTHR=2.*THR;
    Doub absx,rootx,z;
    absx=abs(x);
    rootx=sqrt(absx);
    z=TWOTHR*absx*rootx;
```

```
    if (x > 0.0) {
        besselik(THR,z);
        aio = rootx*ONOVRT*ko/PI;
        bio = rootx*(ko/PI+2.0*ONOVRT*io);
        besselik(TWOTHR,z);
        aipo = -x*ONOVRT*ko/PI;
        bipo = x*(ko/PI+2.0*ONOVRT*io);
    } else if (x < 0.0) {
        besseljy(THR,z);
        aio = 0.5*rootx*(jo-ONOVRT*yo);
        bio = -0.5*rootx*(yo+ONOVRT*jo);
        besseljy(TWOTHR,z);
        aipo = 0.5*absx*(ONOVRT*yo+jo);
        bipo = 0.5*absx*(ONOVRT*jo-yo);
    } else {                          Case x = 0.
        aio=0.355028053887817;
        bio=aio/ONOVRT;
        aipo = -0.258819403792807;
        bipo = -aipo/ONOVRT;
    }
}

Doub Bessel::airy_ai(const Doub x) {
Simple interface returning Ai(x).
    if (x != xo) airy(x);
    return aio;
}
Doub Bessel::airy_bi(const Doub x) {
Simple interface returning Bi(x).
    if (x != xo) airy(x);
    return bio;
}
```

6.6.4 Spherical Bessel Functions

For integer n, spherical Bessel functions are defined by

$$j_n(x) = \sqrt{\frac{\pi}{2x}} J_{n+\frac{1}{2}}(x)$$
$$y_n(x) = \sqrt{\frac{\pi}{2x}} Y_{n+\frac{1}{2}}(x) \tag{6.6.47}$$

They can be evaluated by a call to `besseljy`, and the derivatives can safely be found from the derivatives of equation (6.6.47).

Note that in the continued fraction CF2 in (6.6.3) just the first term survives for $\nu = 1/2$. Thus one can make a very simple algorithm for spherical Bessel functions along the lines of `besseljy` by always recursing j_n down to $n = 0$, setting p and q from the first term in CF2, and then recursing y_n up. No special series is required near $x = 0$. However, `besseljy` is already so efficient that we have not bothered to provide an independent routine for spherical Bessels.

besselfrac.h

```
void Bessel::sphbes(const Int n, const Doub x) {
Sets sphjo, sphyo, sphjpo, and sphypo, respectively, to the spherical Bessel functions j_n(x), y_n(x), and their derivatives j'_n(x), y'_n(x) for integer n (which is saved as sphno).
    const Doub RTPIO2=1.253314137315500251;
    Doub factor,order;
    if (n < 0 || x <= 0.0) throw("bad arguments in sphbes");
    order=n+0.5;
    besseljy(order,x);
    factor=RTPIO2/sqrt(x);
    sphjo=factor*jo;
```